Great! I'll summarize the **whole flow in a simple way so you can remember it easily**.

Think of it as 4 **steps of OAuth2 Login + API Call**.

Simple Memory Version (4 **Steps**)

1️⃣ **User logs in with Keycloak**

Spring Security redirects the user to Keycloak.

Browser → Spring App → Keycloak Login

Config that enables it:

```
.oauth2Login(oauth -> oauth
    .loginPage("/oauth2/authorization/mywebclient")
)
```

After login, Keycloak sends tokens back to Spring:

Access Token

ID Token

2️⃣ **Spring stores the tokens for the user**

Spring Security automatically saves the token in an **OAuth2AuthorizedClient**.

That is why we use:

```
@RegisteredOAuth2AuthorizedClient("mywebclient")
OAuth2AuthorizedClient authorizedClient
```

Spring then gives us the token directly.

3️⃣ **Get the Access Token**

Inside the controller:

```
String accessToken = authorizedClient.getAccessToken().getTokenValue();
```

Important rule to remember:

⚠️ **Access Token → used to call APIs**
⚠️ **ID Token → used for login identity only**

4️⃣ **Call API Gateway with the Access Token**

Your Spring MVC app calls the API gateway like this:

Spring Web Client → API Gateway → Albums Service

Send the token in the header:

```
HttpHeaders headers = new HttpHeaders();
headers.setBearerAuth(accessToken);
```

Result:

Authorization: Bearer <access_token>

Now the API Gateway trusts the request because it sees the valid token.

Full Flow (Easy Diagram)

1 Browser
  |
  ▼
2 Spring Web Client
  |
  ▼
3 Redirect to Keycloak
  |
  ▼
4 User Login
  |
  ▼
5 Keycloak returns Tokens
  |
  ▼
6 Spring stores OAuth2AuthorizedClient
  |
  ▼
7 Controller gets Access Token
  |
  ▼
8 Call API Gateway with Bearer Token
  |
  ▼
9 API Gateway → Albums Service

|
▼

🔟 HTML returned to browser

---

3 **Key Things to Always Remember**

1️⃣ **Use @RegisteredOAuth2AuthorizedClient**

Instead of manually loading the client.

@RegisteredOAuth2AuthorizedClient("mywebclient")

OAuth2AuthorizedClient authorizedClient

2️⃣ **Use Access Token for APIs**

Correct:

Authorization: Bearer ACCESS_TOKEN

Wrong:

Authorization: Bearer ID_TOKEN ❌

3️⃣ **Registration ID vs Client ID**

Example:

registration id = mywebclient

client id = photo-app-webclient

Use **registration id** here:

@RegisteredOAuth2AuthorizedClient("mywebclient")

One Sentence to Remember Everything
👉 **Login with Keycloak → Spring stores tokens → Controller gets Access Token → send it as Bearer token to API Gateway**.

If you want, I can also give you a **very clean architecture diagram of your project** (**Keycloak** + **WebClient** + **Gateway** + **Resource Server**) that makes this even easier to remember.